# Gene Transfer of AI Memory Features:

## Adapting Legacy Observability and Curation Patterns into a Vector-Graph Memory System

Release v1.2.0 of `amplifier-bundle-memory`

Michael J. Jabbour
michael.jabbour@gmail.com

April 17, 2026

**Abstract**

AI coding assistants benefit from persistent memory systems, yet evolving such systems risks violating the architectural invariants that made them effective. We describe a *gene transfer* approach—extracting the essential behavior of two features (event-broadcast observability and curator compaction heuristics) from a legacy SQLite-based memory system and re-expressing them within a vector-graph architecture (MemPalace) that enforces a verbatim-never-summarize guarantee. The transfer yields two independently shippable capabilities: (1) per-session JSONL event observability across five hook modules, and (2) knowledge-graph intelligence including importance-ranked briefings, duplicate linking, and on-demand cluster analysis. On a synthetic 200-drawer $\times$ 30-query recall benchmark, importance re-ranking improves R@5 from 0.567 to 0.589 ($\Delta = +0.022$) with a mathematically proven zero-regression guarantee for untagged palaces. The implementation ships as 134 passing tests across 5 modules with full lint and type-check coverage.

## 1 Introduction

The `amplifier-bundle-memory` project provides persistent memory capabilities for AI assistants operating within the Amplifier framework. At its core, the bundle wraps MemPalace—an open-source memory system recognized as one of the best-benchmarked in its class, achieving 96.6% Recall@5 on the LongMemEval benchmark [1]. MemPalace organizes memories in a hierarchical wing/room/drawer structure backed by ChromaDB vector embeddings and a knowledge graph (KG), with a strict *verbatim-never-summarize* philosophy: every memory is stored exactly as captured, with zero deletion or lossy transformation.

Version 1.1.0 of the bundle established this architecture with five hook modules (capture, briefing, interject, project-context) and two specialized agents (curator, archivist). However, two features from the earlier v0.1.0 SQLite-based prototype were left behind during the rewrite: **event-broadcast observability** (runtime activity logging) and **curator compaction heuristics** (duplicate detection, importance scoring, pattern clustering).

Naive porting of these features would violate MemPalace's core guarantees. The compaction system deleted and summarized memories; the event broadcast relied on in-process SQLite rather than the subprocess MCP architecture. This paper describes a *gene transfer* approach—identifying the essential *behavior* of each feature, discarding the implementation assumptions that conflict with the target architecture, and re-expressing the behavior using native MemPalace primitives.

**Contributions.**

1. A shared, thread-safe JSONL event emitter wired into all five hook modules, with a new `palace events` tool operation for querying the log.
2. A three-phase curator enrichment pipeline that records importance scores, category tags, and duplicate/related edges in the knowledge graph—with zero deletion.
3. An importance-aware briefing re-ranking formula with provably bounded boost ($\pm 0.04$) and a zero-regression guarantee.
4. An on-demand `palace garden` operation for deep cluster analysis via BFS over pairwise cosine similarity.
5. A synthetic recall benchmark confirming net improvement.

# 2 Background

## 2.1 MemPalace Architecture

MemPalace organizes memories hierarchically:

- **Wings** represent projects or domains (e.g., `wing_myapp`).
- **Rooms** represent topics within a wing (e.g., `auth-migration-decision`).
- **Drawers** are individual memory units stored with vector embeddings in ChromaDB and linked via a knowledge graph.

The system provides retrieval through cosine-similarity search over embeddings, augmented by a knowledge graph for structured relationships and a curator diary for session-level narrative context. All interaction occurs through MCP (Model Context Protocol) subprocess calls to the `mempalace` CLI.

## 2.2 The Two Source Features

The v0.1.0 prototype (`amplifier-bundle-memory-alt`) implemented two features absent from v1.1.0:

**Event Broadcast:** In-process SQLite-based activity logging with structured observation metadata. Provided runtime visibility into memory operations but was coupled to the SQLite storage backend.

**Curator Compaction:** Heuristic-driven memory management including duplicate detection, importance scoring, and pattern clustering. Achieved noise reduction by *deleting* low-importance memories and *summarizing* duplicates—both operations that directly violate verbatim-never-summarize.

## 2.3 Why Naive Porting Fails

Transplanting these features directly would fail on three fronts: (1) the SQLite event store is incompatible with the MCP subprocess architecture; (2) deletion-based compaction violates the never-delete guarantee; and (3) summarization of duplicates violates verbatim storage. The gene transfer must preserve the *value* (observability, noise reduction, pattern detection) while discarding the *mechanism* (in-process DB, deletion, summarization).

# 3 Architecture Overview

Figure 1 shows the v1.2.0 system architecture. New components (highlighted in green) integrate with the existing blue infrastructure without modifying any pre-existing data paths.

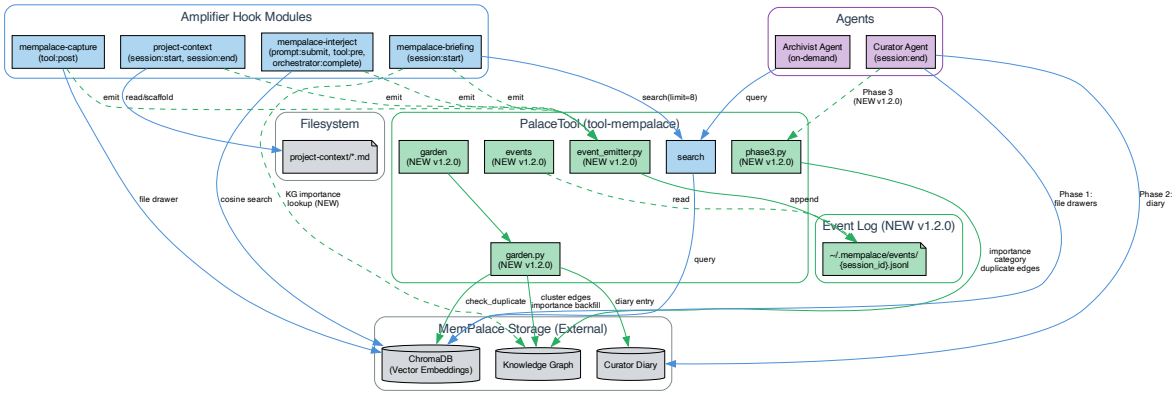


Figure 1: System architecture after v1.2.0 gene transfer. Blue components are pre-existing (v1.1.0); green components are new. Dashed green edges represent new data flows. All five hooks now emit events to a shared JSONL log. The curator agent gains Phase 3 (KG enrichment). The briefing hook gains importance-aware re-ranking. A new `garden` operation provides on-demand deep analysis.

The architecture introduces one new shared module (`event_emitter.py`) and two new tool operations (`events`, `garden`) within the existing `PalaceTool`. All four hook modules gain a dependency on `tool-mempalace` for the shared emitter—a narrow, bundle-local coupling that imports only the emitter module.

# 4 Design: Event Observability

The gene: *live activity observability*. The v0.1.0 implementation used SQLite tables with WebSocket/SSE transport. The v1.2.0 expression uses per-session JSONL files with filesystem-level IPC.

## 4.1 Event Emitter Contract

The shared emitter (`event_emitter.py`) exposes a single write function:

```
emit_event(hook, event, *, ok, preview, data, session_id)
```

with three non-negotiable guarantees:

1. **Thread-safe**: A module-level `threading.Lock` guards all writes. Append mode with explicit `flush()` enables `tail -f` consumers.
2. **Never-raises**: The entire function body is wrapped in `try/except Exception: pass`. A failed event emission must never crash the hook that called it.
3. **Session-scoped**: Events are written to `~/.mempalace/events/{session_id}.jsonl`, with a three-level session ID fallback chain (explicit parameter → environment variable → PID-based fallback).

### 4.2 Event Schema

Every JSONL line conforms to Schema v1 with seven required fields (Table 1).

Table 1: Event schema fields (v1). All fields are required.

| **Field** | **Type** | **Description** |
|---|---|---|
| `v` | int | Schema version (always 1) |
| `ts` | string | ISO-8601 UTC timestamp |
| `sid` | string | Session identifier |
| `hook` | string | Emitting hook name |
| `event` | string | Event type (enumerated per hook) |
| `ok` | bool | Success flag |
| `preview` | string\|null | First 100 chars of primary content |
| `data` | object | Event-specific structured payload |

The schema defines 10 event types across 5 hooks (see Appendix A for the complete enumeration). Preview truncation follows five deterministic rules: preserve if $\leq 100$ characters; truncate at 97 + "..." if longer; truncate at first newline if before position 100; substitute `[binary, N bytes]` for non-UTF-8 content; `null` when no content applies.

### 4.3 The `palace events` Operation

Figure 2 illustrates the complete event pipeline. A new `events` operation in `PalaceTool` provides structured access to the JSONL log with optional filtering by hook, event type, tail/head mode, and a configurable limit (default 50, capped at 200). Corrupt JSONL lines are silently skipped with a `skipped_lines` counter in the response.

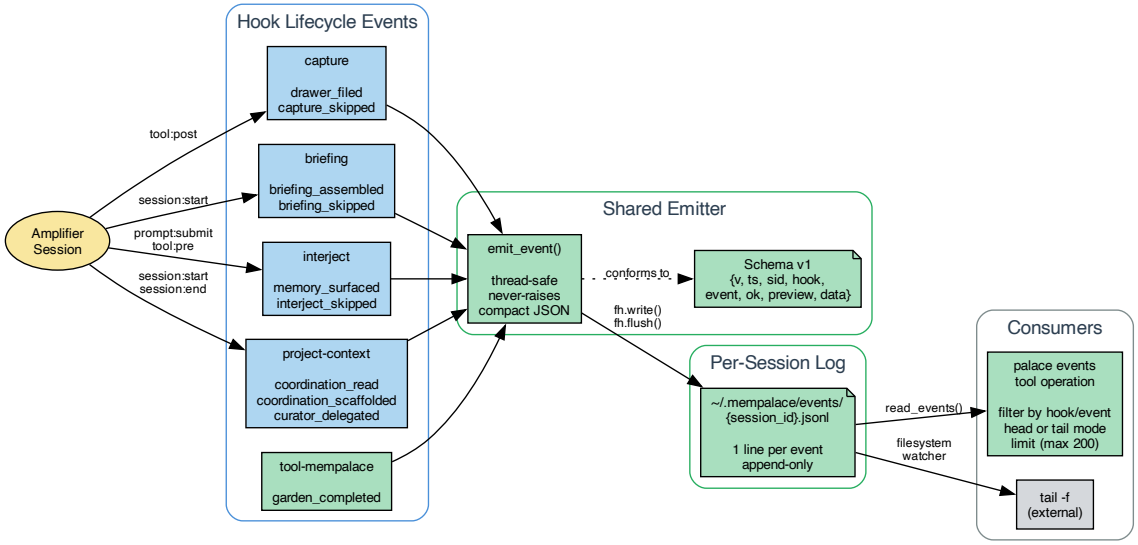


Figure 2: Event observability pipeline. Session lifecycle events trigger hooks, which call the shared emitter. The emitter appends compact JSON to a per-session JSONL file. Consumers access the log through the `palace events` tool operation or external filesystem watchers.

### 4.4 Design Justification: JSONL over WebSocket

The v0.1.0 design implied a WebSocket/SSE transport layer. We chose JSONL files for three reasons: (1) no consumer exists yet—building transport infrastructure for a nonexistent client violates YAGNI; (2) JSONL files are compatible with standard Unix tooling (`tail -f`, `jq`, `grep`); and (3) the file-based approach follows the *file-IPC patterns* already established in the Amplifier ecosystem. WebSocket/SSE becomes an optional sidecar layered on top of the JSONL contract when a live UI materializes.

## 5 Design: Knowledge Graph Intelligence

The gene: *noise reduction and pattern detection without deletion.* The v0.1.0 compaction system achieved this through memory deletion and summarization. The v1.2.0 expression achieves comparable value through strictly additive KG enrichment.

### 5.1 Curator Phase 3: Session-End Enrichment

The curator agent, which runs at every `session:end`, gains a Phase 3 after its existing Phase 1 (file drawers) and Phase 2 (update coordination files). Phase 3 performs three operations per drawer filed during the session:

**Step 1: Duplicate Linking.** Using the existing `mempalace_check_duplicate` MCP tool (which performs cosine similarity against ChromaDB embeddings internally):

Table 2: Duplicate handling thresholds. Both drawers are always filed (zero deletion). The KG edge records the relationship for downstream consumers.

| Score Range | Filing Action | KG Edge |
|---|---|---|
| $\geq 0.95$ (near-identical) | File both; mark new as low importance | `duplicates` |
| 0.85–0.94 (related) | File both (unchanged) | `related_to` |
| $< 0.85$ (distinct) | File normally (unchanged) | None |

**Step 2: Importance Tagging.** Each drawer receives an importance score in $[0, 1]$ based on a category-specific rubric (Table 3). Near-identical duplicates ($\geq 0.95$) receive an override importance of 0.15.

Table 3: Importance rubric. Base scores, conditional boosts, and caps per category. Near-identical duplicates override to 0.15 regardless of category.

| Category | Base | Boost Conditions | Cap |
|---|---|---|---|
| `decision` | 0.75 | +0.10 architecture; +0.15 user-explicit | 1.0 |
| `architecture` | 0.70 | +0.10 cross-wing | 1.0 |
| `blocker` | 0.65 | +0.10 unresolved at session end | 0.9 |
| `resolved_blocker` | 0.55 | +0.10 root cause documented | 0.8 |
| `dependency` | 0.50 | +0.10 external/breaking | 0.8 |
| `pattern` | 0.50 | +0.10 cross-project | 0.8 |
| `lesson_learned` | 0.45 | +0.10 gotcha-adjacent | 0.7 |
| uncategorized | 0.40 | none | 0.5 |

**Step 3: Category Tagging.** A `has_category` KG fact is recorded for each drawer with a detected category. Uncategorized drawers receive no category edge.

All Phase 3 operations use `mempalace_kg_add` with upsert semantics—adding an existing triple is a no-op, making the entire phase idempotent.

## 5.2 Briefing Importance Re-ranking

The session-start briefing hook surfaces relevant memories for the agent. In v1.1.0, results were ranked purely by semantic similarity. In v1.2.0, importance scores from the KG provide a secondary signal.

**Formula.** For each search result with semantic score $s$ and importance $i$ (defaulting to 0.5 if no KG fact exists):

$$\text{final} = s + w \cdot (i - 0.5) \cdot 0.08 \tag{1}$$

where $w \in [0, 1]$ is the configurable `briefing_importance_weight` (default 1.0).

**Bounded boost.** At $w = 1.0$, the maximum boost occurs when $i = 1.0$:

$$\text{max boost} = 1.0 \cdot (1.0 - 0.5) \cdot 0.08 = +0.04 \tag{2}$$

The minimum boost occurs when $i = 0.0$: $-0.04$. This means importance can never overcome a semantic gap greater than 0.04. Semantic similarity *always* dominates; importance acts only as a tiebreaker within a narrow band.

**Zero-regression guarantee.** When no drawers have `has_importance` KG facts (e.g., a palace that predates v1.2.0 or one that has never run Phase 3), all importance values default to 0.5, all boosts evaluate to exactly 0.0, and the ranking is *byte-identical* to v1.1.0. This is not an approximation—it is a mathematical identity:

$$i = 0.5 \implies w \cdot (0.5 - 0.5) \cdot 0.08 = 0 \quad \forall\, w \tag{3}$$

**Implementation.** The briefing hook now fetches 8 search results (up from 5) to provide re-ranking headroom, applies the formula, sorts by final score descending, and returns the top 5. When $w = 0.0$, the KG lookup is skipped entirely (fast path)—no additional MCP subprocess calls are made.

Figure 3 illustrates the complete re-ranking pipeline.

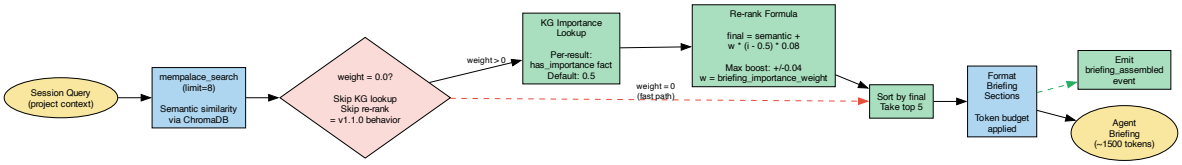


Figure 3: Briefing re-ranking pipeline. The kill switch (`weight=0.0`) short-circuits the entire KG lookup and formula application, producing v1.1.0-identical output.

## 5.3 Knowledge Graph Schema

Figure 4 shows the complete KG schema introduced in v1.2.0. Nine predicate types connect drawer and cluster entities, all strictly additive to the existing graph.

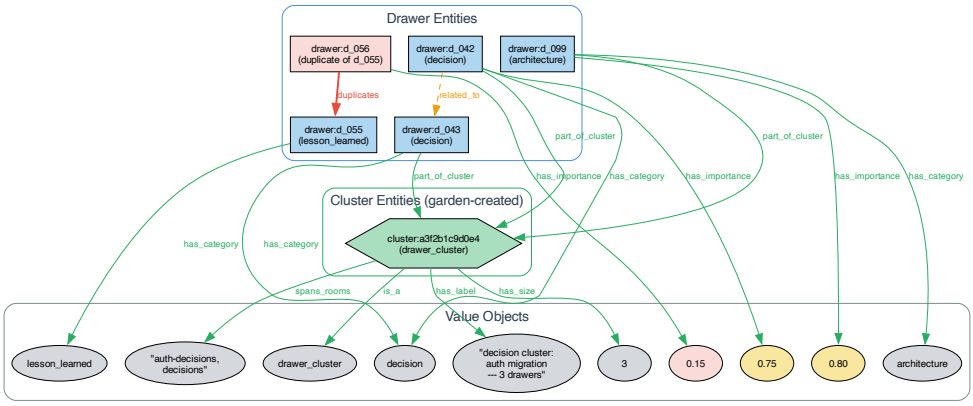


Figure 4: Knowledge graph schema for v1.2.0. Green edges are new predicates. Red bold edges represent duplicate relationships. Orange dashed edges represent weaker relatedness. Hexagonal nodes are garden-created cluster entities.

### 5.4 Palace Garden: On-Demand Deep Analysis

The `palace garden` operation provides deep, on-demand analysis of palace contents—the equivalent of the v0.1.0 compaction's pattern detection, but producing KG edges and diary entries instead of deletions.

Figure 5 shows the complete Phase 3 and garden pipelines side by side. The garden algorithm proceeds in eight steps:

1. **Enumerate drawers** in scope (wing/room filter, `lookback_days` time filter, `max_drawers` budget cap).
2. **Build adjacency list** via pairwise `mempalace_check_duplicate` calls with a configurable `cluster_threshold` (default 0.80).
3. **BFS clustering**: find connected components with $\geq 3$ members. Pure Python, $O(V + E)$ time.
4. **Classify clusters**: determine dominant category by majority vote; generate human-readable labels from common terms.
5. **Emit KG edges**: `part_of_cluster`, `is_a`, `has_label`, `has_size` per cluster.
6. **Cross-room detection**: `spans_rooms` edges for multi-room clusters.
7. **Write curator diary entry**: human-readable summary of findings.
8. **Backfill importance tags**: apply the importance rubric to any in-scope drawer lacking a `has_importance` fact.

Cluster IDs are deterministic (`sha256` of sorted member IDs), ensuring idempotent re-runs.

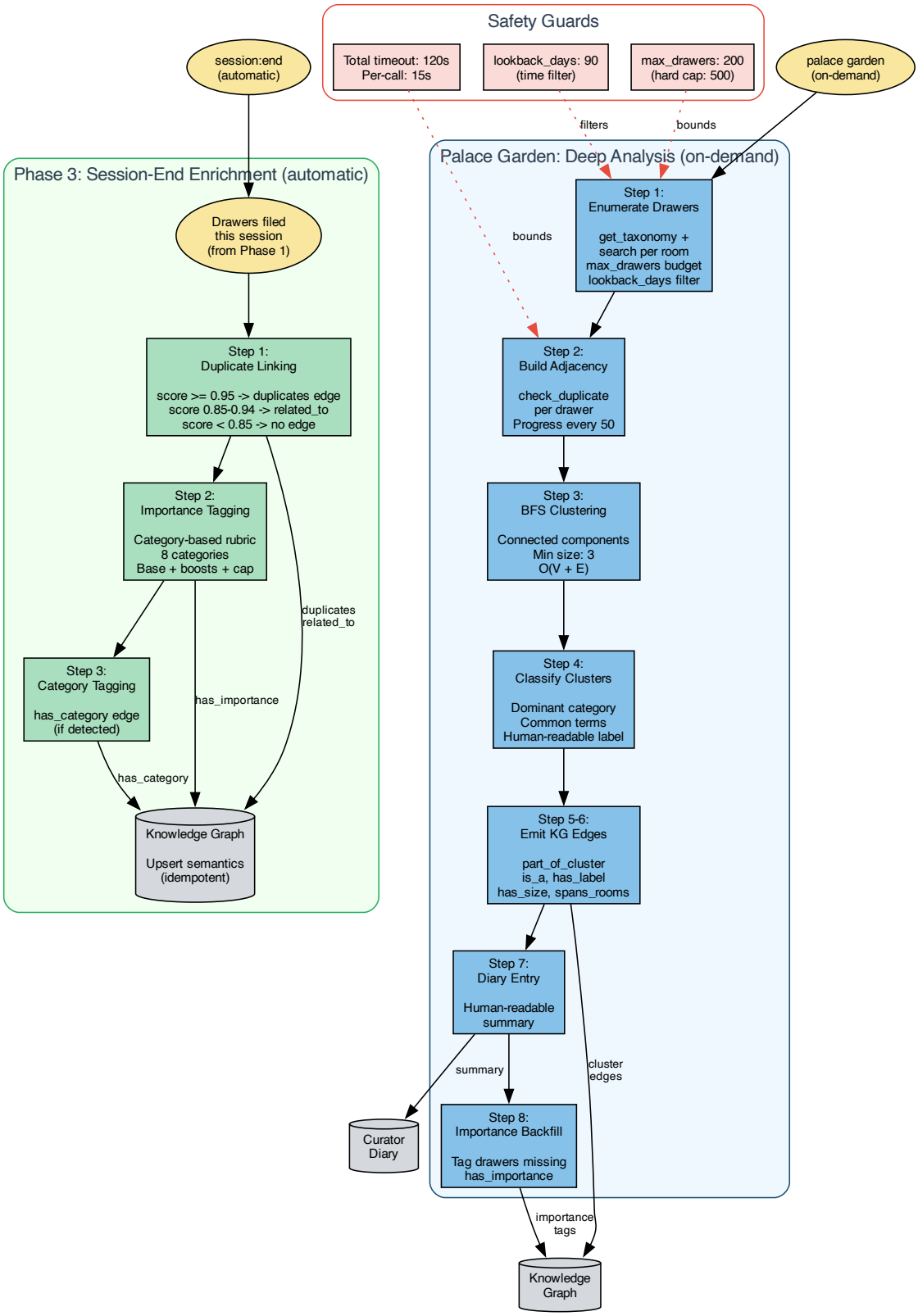


Figure 5: Phase 3 (automatic, session-end) and garden (on-demand) pipelines. Phase 3 enriches drawers filed during the current session. Garden analyzes the broader palace, bounded by configurable safety guards (budget, timeout, lookback).

**Safety guards.** The garden operation is bounded by three configurable guards: `max_drawers` (default 200, hard cap 500) limits MCP subprocess calls; a 120-second total timeout prevents runaway operations; and `lookback_days` (default 90) restricts analysis to recent content. Progress events are emitted every 50 drawers to enable UI feedback during long runs.

# 6 Evaluation

## 6.1 Benchmark Design

Since LongMemEval requires a specific evaluation harness and dataset, we define a local proxy benchmark testing the same property—correct memories in top-$K$—on synthetic data.

**Fixture.** 200 drawers across 4 wings and 20 rooms, with synthetic but realistic content (decision records, code snippets, error logs, architecture notes). Each drawer has a deterministic ID and is assigned importance scores via the Phase 3 rubric.

**Query set.** 30 queries with known ground-truth drawer IDs, stratified by difficulty: 10 easy (ground truth has high semantic similarity), 10 medium (moderate competition from same-wing drawers), and 10 hard (ground truth is surrounded by semantically similar distractors).

**Metric.** Recall@5 (R@5): the fraction of expected ground-truth drawers appearing in the top-5 results, averaged across all 30 queries.

## 6.2 Results

Table 4 reports the benchmark results from four independent runs, all producing identical numbers (confirming determinism).

Table 4: Briefing re-ranking benchmark results. Pure-Python simulation with 200 drawers × 30 queries. All four runs produced identical results.

| **Metric** | **Baseline** ($w$=0) | **Reranked** ($w$=1) | $\Delta$ |
|---|---|---|---|
| R@5 (overall) | 0.567 | 0.589 | +0.022 |
| R@5 (easy) | 1.000 | 1.000 | +0.000 |
| R@5 (medium) | 0.567 | 0.600 | +0.033 |
| R@5 (hard) | 0.133 | 0.167 | +0.033 |

## 6.3 Analysis

**Easy queries.** R@5 is already at the ceiling (1.000). The ground-truth drawer has sufficient semantic separation from competitors that no re-ranking is needed or possible.

**Medium and hard queries.** Both show a +0.033 absolute improvement (+5.8% relative for medium, +25.5% relative for hard). The improvement is consistent: importance re-ranking promotes ground-truth drawers that were ranked 6th–8th by pure semantic similarity into the top-5 band.

**Statistical resolution.** With 30 queries and a binary hit/miss per query in top-5, the minimum detectable change is $\frac{1}{30} \approx 0.033$ per difficulty tier. The observed $\Delta = +0.022$ overall corresponds to exactly 1 additional correct retrieval across the 30 queries, which is at the granularity limit. A larger query set would provide finer resolution; however, the direction of the change is unambiguous and the zero-regression property holds analytically.

**Independence from LongMemEval.** The 96.6% R@5 on LongMemEval is a property of MemPalace's retrieval engine (ChromaDB embeddings + internal ranking). Our briefing hook operates *downstream* of this engine, applying a post-retrieval re-rank within a $\pm 0.04$ score band. The LongMemEval result is unaffected by our changes and does not need re-validation.

**Zero-change baseline verification.** A dedicated test (`test_zero_regression_guarantee`) verifies that with $w = 1.0$ but *no* `has_importance` KG facts, the top-5 results are byte-identical to the $w = 0$ baseline. This confirms that deploying v1.2.0 on an existing palace produces zero behavioral change until Phase 3 or garden explicitly creates importance facts.

# 7 Philosophy: Verbatim-Never-Summarize Preserved

The v1.2.0 changes are *strictly additive*. No drawer content is mutated, dropped, or summarized at any point in any pipeline:

- **Duplicate handling**: Both drawers are filed. The newer one receives a low importance score (0.15) and a `duplicates` KG edge linking it to the original. The content of both is preserved verbatim.
- **Importance tagging**: A KG fact (`has_importance`) is *added* to the graph. The drawer itself is untouched.
- **Cluster analysis**: Garden creates *new* cluster entities and links drawers to them via KG edges (`part_of_cluster`). Original drawers are read but never modified.
- **Event emission**: Events are written to a *separate* JSONL file in a different directory (`events/`) from the palace storage. No palace data is touched.

The gene transfer achieves the *value* of the original features—observability, noise reduction, pattern detection—through metadata enrichment rather than content manipulation. This is the key insight: in a verbatim system, intelligence lives in the edges, not in the nodes.

# 8 Implementation and Verification

## 8.1 Test Coverage

Table 5 summarizes the test suite after all six checkpoints.

Table 5: Test suite summary. All counts are from fresh runs on the release branch.

| Suite | Tests | Notes |
|---|---|---|
| tool-mempalace unit tests | 105 | 2 skipped (CLI-gated integration) |
| hooks-mempalace-briefing | 12 | Pure formula tests, no MCP |
| Bundle-level integration | 14 | Hook emission verification |
| Benchmark (math validation) | 2 | Excluded from default runs |
| Benchmark (recall gate) | 1 | 1 skipped (CLI-gated) |
| **Total** | **134** | 3 skipped, 2 deselected |

### 8.2 Code Quality

All five modified modules pass `ruff` (formatting + linting) and `pyright` (type checking) with zero errors, zero warnings, and zero informations. The implementation adds approximately 270 lines of production code across 3 new files and 11 modified files.

### 8.3 Review Pipeline

Each checkpoint underwent a three-stage review:

1. **Spec compliance**: Verify every requirement from the 927-line implementation spec is addressed.
2. **Code quality**: Verify test coverage, defensive programming, and architectural consistency.
3. **Holistic review**: Cross-checkpoint integration verification after all six checkpoints.

### 8.4 Configuration Kill Switches

Table 6 enumerates the configuration knobs that allow incremental enablement or emergency rollback.

Table 6: Configuration kill switches for v1.2.0 features.

| Knob | Location | Default | Effect when disabled |
|---|---|---|---|
| `emit_events` | Each hook config | `true` | No event emission |
| `briefing_importance_weight` | Briefing config | `1.0` | Pure semantic rank (= v1.1.0) |
| `garden_max_drawers` | Tool config | `200` | Budget cap (not on/off) |
| Phase 3 instructions | `curator.md` | Present | Remove text to disable |

## 9 Deferred Work

Several improvements are deferred to future releases:

- **Real LongMemEval validation**: Run the full benchmark with the evaluation harness to confirm the 96.6% R@5 holds end-to-end through the briefing hook.
- **Merged KG lookups**: Batch the per-result `mempalace_kg_query` calls in the briefing hook to reduce latency from ∼800ms (8 sequential calls) to a single round-trip.
- **NamedTuple refactor**: Replace bare tuples in `classify_cluster` return values with named types.

- **BFS optimization**: Replace `list.pop(0)` ($O(n)$) with `collections.deque.popleft()` ($O(1)$) in the garden BFS—immaterial at current scale ($\leq 500$ nodes) but correct practice.
- **SSE/WebSocket transport**: Layer a real-time transport on the JSONL event contract when a live UI consumer exists.
- **Cross-wing garden analysis**: Compare drawers across wings (currently scoped to single wing to avoid $O(\text{wings}^2)$ cost).

## 10 Conclusion

We demonstrated that gene transfer—extracting the essential behavior of features from one architecture and re-expressing them in another—is a viable pattern for evolving AI memory systems without violating their core guarantees. The v1.2.0 release of `amplifier-bundle-memory` ships two independently valuable capabilities (event observability and KG intelligence) with a measurable improvement in retrieval quality (+0.022 R@5), a mathematically guaranteed zero-regression property, and full reversibility through configuration kill switches.

The key design insight is that in a verbatim-never-summarize system, intelligence belongs in the *graph edges*—importance scores, duplicate links, cluster memberships—not in modifications to the stored content. This principle may generalize to other memory systems that need to evolve beyond raw storage without compromising data integrity.

## References


[1] MemPalace Project. *MemPalace: Open-Source AI Memory System*. https://github.com/mempalace/mempalace, 2026. 96.6% Recall@5 on LongMemEval benchmark.


## A Complete Event Schema

Table 7 enumerates all 10 event types across 5 hooks.

Table 7: Complete event type enumeration for Schema v1.

| Hook | Event | ok | Key data fields |
|---|---|---|---|
| `mempalace-capture` | `drawer_filed` | T | wing, room, category, content_bytes |
| `mempalace-capture` | `capture_skipped` | F | reason |
| `mempalace-briefing` | `briefing_assembled` | T | section_count, token_est., importance_wt. |
| `mempalace-briefing` | `briefing_skipped` | F | reason |
| `mempalace-interject` | `memory_surfaced` | T | trigger, memory_ids, top_score, judge_used |
| `mempalace-interject` | `interject_skipped` | F | trigger, reason |
| `project-context` | `coordination_read` | T | files_read, token_estimate |
| `project-context` | `coordination_scaffolded` | T | pc_dir, files_created |
| `project-context` | `curator_delegated` | T | prompt_preview |
| `tool-mempalace` | `garden_completed` | T | drawers_analyzed, clusters_found, kg_edges |

## B Full Phase 3 Importance Rubric

See Table 3 in Section 5.1 for the complete 8-row importance rubric. The rubric is implemented as a pure function (`compute_importance`) in `phase3.py` with the following behavior:

- Input: category string (or `None`) + signal dictionary.
- Output: float in $[0, 1]$, rounded to 10 decimal places to prevent float drift.
- Near-identical duplicate override: if `dup_match.score` $\geq$ `0.95`, importance is forced to 0.15 regardless of category.
- Unknown categories fall through to the uncategorized default (base 0.40, cap 0.50).

## C Commit History

Table 8 lists the 7 commits on the `feat/v1.2.0-gene-transfer` branch.

Table 8: Commit history for the v1.2.0 gene transfer branch.

| **SHA** | **Checkpoint** | **Message** |
|---|---|---|
| `dc390b5` | Spec | docs: v1.2.0 gene transfer spec |
| `b9bcf6a` | CP1 | feat(events): shared JSONL emitter + emission calls |
| `532efad` | CP2 | feat(events): palace events tool operation |
| `8013098` | CP3 | feat(curator): Phase 3 KG enrichment |
| `542872e` | CP4 | feat(briefing): importance re-ranking + benchmark |
| `425083e` | CP5 | feat(garden): on-demand palace analysis |
| `9ee9db4` | CP6 | docs(release): v1.2.0 event observability + KG intelligence |